GOSIM

GOSIM Shenzhen 2026

LMCache
统一 KV Cache 管理层

架构演进与生产实践





郑春晓

LMCache Maintainer & 核心架构师

- 项目第五大贡献者，累计合并 130+ PR
- LMCache 核心 Maintainer
- vLLM Contributor
- SGLang Contributor
- 毕业以来专注于大数据存储和KV Cache相关存储

CONTENTS

GOSIM Shenzhen 2026

Part 01 LMCache 架构演进

Part 02 Multi-Process 模式架构

Part 03 Multi-Process 模式性能优势

Part 04 可插拔扩展与多平台

Part 05 生产部署与可观测性

Part 06 社区与未来方向



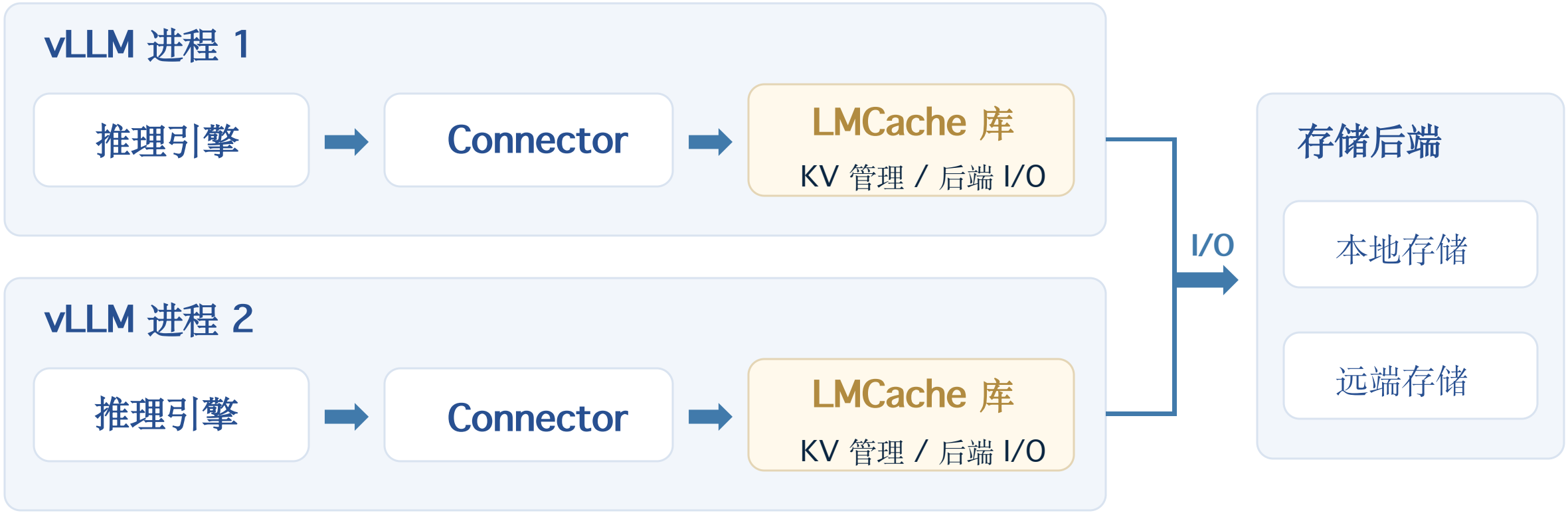
CONTENTS

01 LMCache 架构演进



01 LMCache 架构演进

In-process / Library 模式



KV 管理库与推理引擎位于同一进程

In-process：多实例下的工程问题

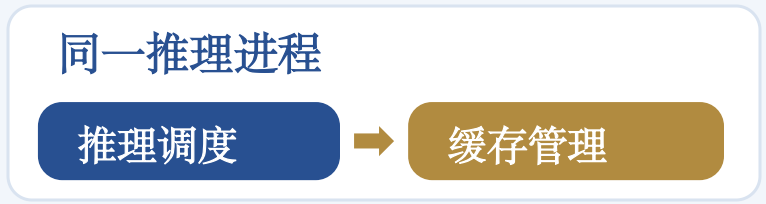
01 LMCache 架构演进



01 多实例状态分散，存在冗余开销

相同 KV 可能重复 offload，CPU 容量按进程占用

部分路径还需跨 rank 分发，占用额外传输带宽



02 运行时耦合，缓存工作可能干扰推理

Python 缓存逻辑可能与引擎争用同一 GIL

I/O 提交、回调与同步等待，可能进入请求关键路径



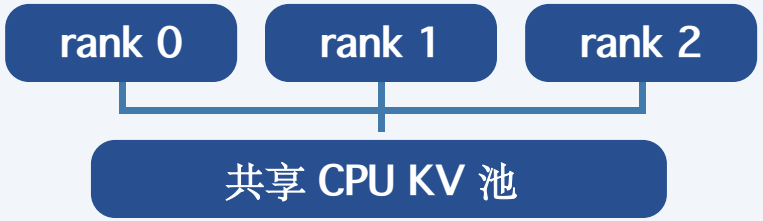
03 生命周期耦合，管理边界不清晰

引擎退出后，进程内缓存状态与句柄需要重建

容量规划、监控和故障定位分散在各推理实例中

优化方向：共享缓存与独立服务

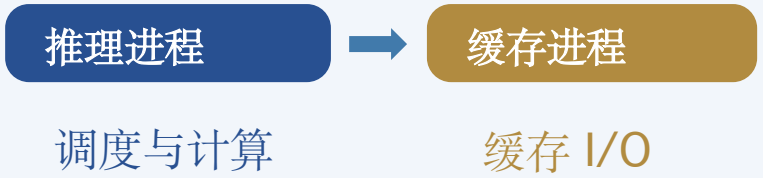
01　LMCache 架构演进



01　集中管理节点内的共享 KV

以共享 CPU 池统一规划容量，减少兼容 KV 的重复驻留

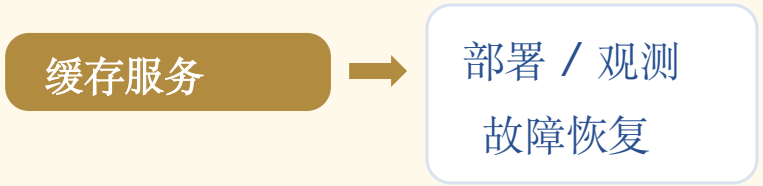
统一组织加载与分发，避免不必要的重复搬运



02　分离缓存运行时，独立推进 I/O

引擎与缓存服务分进程，通过 IPC 协作并同步完成状态

按操作隔离队列，降低慢 I/O 对关键读取路径的干扰



03　建立独立服务的部署与恢复边界

缓存容量与引擎副本分别规划，集中暴露监控与管理接口

明确注册、重连、降级及恢复验证流程

CONTENTS

02 Multi-Process 模式架构



MP 模式架构

02 Multi-process 模式架构

推理实例

vLLM Scheduler
LMCacheMPConnector

vLLM Worker 0
LMCacheMPConnector

vLLM Worker 1
LMCacheMPConnector

LMCache 实例

LMCache Server

独立进程 / 独立生命周期

- CPU L1
- L2 Adapters
- Prometheus
- HTTP / CLI

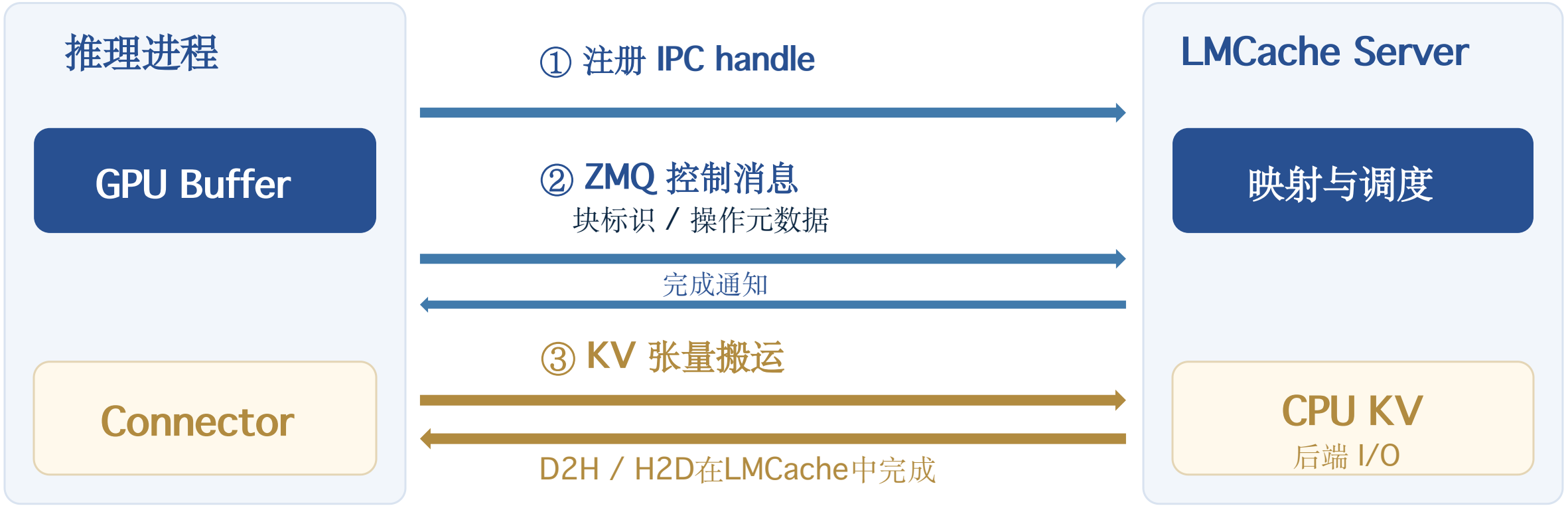
独立的 KV cache 管理服务: 每节点一个 daemon,
通过 CudaIPC 共享 vLLM GPU buffer

- 无干扰
- 全局视野
- 可被管理

将 KV Cache 管理从推理引擎中解耦，是提升大模型推理服务性能和稳定性的核心设计思想

MP 模式通信

02 Multi-process 模式架构



1. 零拷贝共享 (Zero-Copy)
2. 轻量元数据通信
3. 本地化高效传输

MP 模式架构收益

02 Multi-process 模式架构



01 性能与复用

节点内共享，隔离 Python 运行时

多级缓存并行加载与拼接

02 管理与运维

集中监控与管理接口

独立故障边界与恢复验证

03 扩展与集成

引擎 / 存储 / 设备插件

沿接口边界独立演进

架构革新：将 KVCache 管理从推理进程中剥离，实现计算与存储的物理隔离，彻底解决资源争抢问题。

CONTENTS

03 Multi-Process 模式性能优势

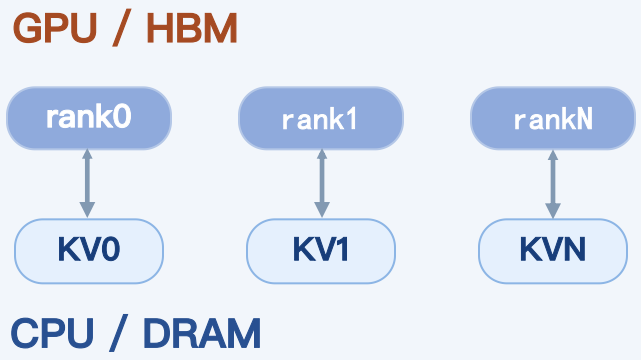


节点内 CPU KV 共享：以 MLA 为例

02 Multi-process 模式性能优势

01 In-process：进程内缓存

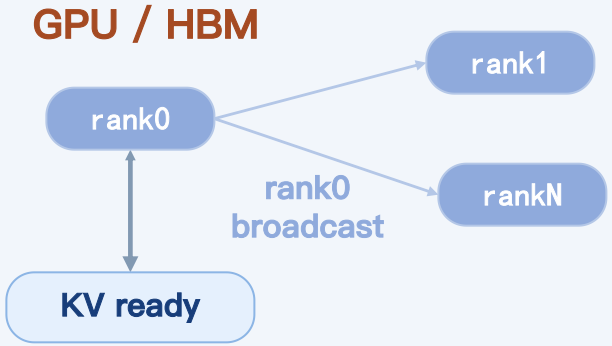
每个 TP Worker 内嵌 KV 库
相同 KV 在 CPU 内存重复 offload
每个 rank 保留一份副本



⚠ 数据冗余存储

02 In-process：跨 Rank 分发

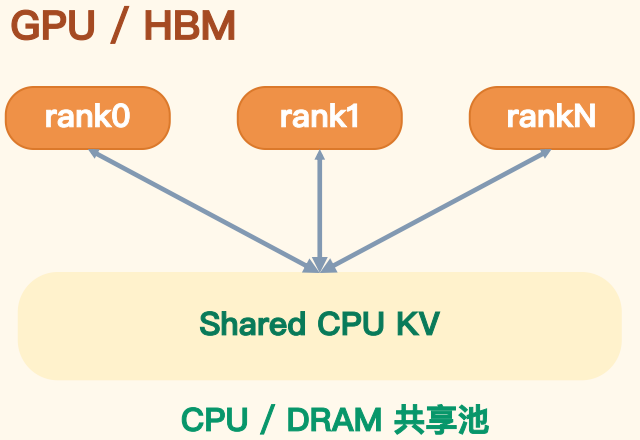
某个 Worker 先 load KV
再经 rank0 向其它 rank 广播
额外占用 GPU 带宽与计算流



⚠ GPU 广播性能损失

03 LMCache MP：单机共享池

独立 LMCache 进程维护共享池
所有 Worker 访问同一份 CPU KV
无需重复 offload 与 GPU 广播



√ 单份共享，免 GPU 广播

进程隔离：消除GIL开销

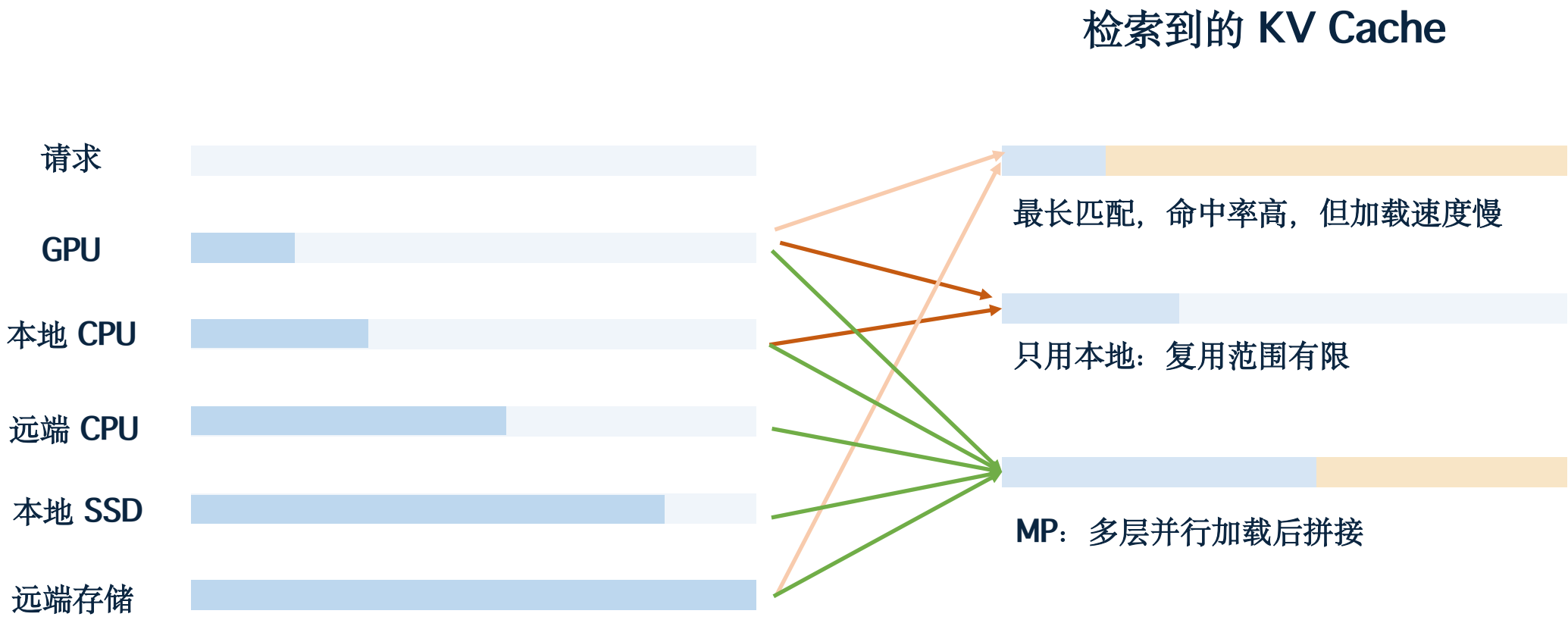
02 Multi-process 模式性能优势



共享物理资源：CPU / 内存带宽 / I/O 带宽

多级缓存的并行加载与拼接

02 Multi-process 模式性能优势



按分段来源组织加载，关注最慢段与同步等待对请求延迟的影响

CONTENTS

04 可插拔扩展与多平台



可插拔扩展的三个接口边界

01

Connector：引擎接入

对接调度、查找、保存与加载，协调异步完成和失败处理

02

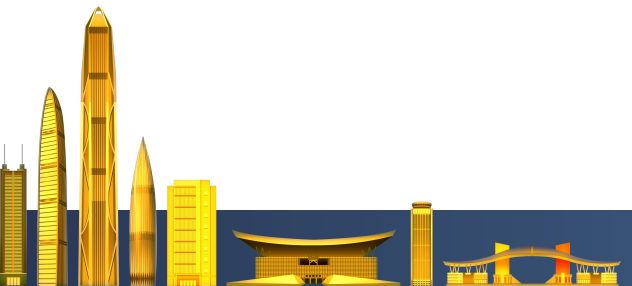
L2 Adapter：存储后端

统一异步 I/O 接口，插件独立维护，保留后端实现差异

03

Device Platform：设备能力

封装设备描述、内存操作、同步与可用传输路径



推理引擎 Connector 与模型布局适配

03 可插拔扩展与多平台

vLLM
KV Connector

SGLang
Connector

TRT-LLM
MP / KV Connector

LMCache
统一 KV Cache 管理层

Connector 职责

调度侧与 Worker 协作
查找、保存、加载及完成事件

模型布局

EngineKVFormat / KV groups
模型、精度与分片兼容性

引擎变化优先适配 Connector，模型变化校验 KV 格式与输出

L1 / L2 分层与存储插件

03 可插拔扩展与多平台

LMCache

L1 热数据层

CPU 内存 / GDS

Store / Prefetch / Load

L2 Adapter

统一异步接口
隔离后端差异

Amazon S3

s3

Object store

FS

filesystem

NIXL Storage

GDS / POSIX / HF3FS

Native Connector

C++ fast path

Native Connector 后端实现

Redis

FS Native

Mooncake

Mooncake Store

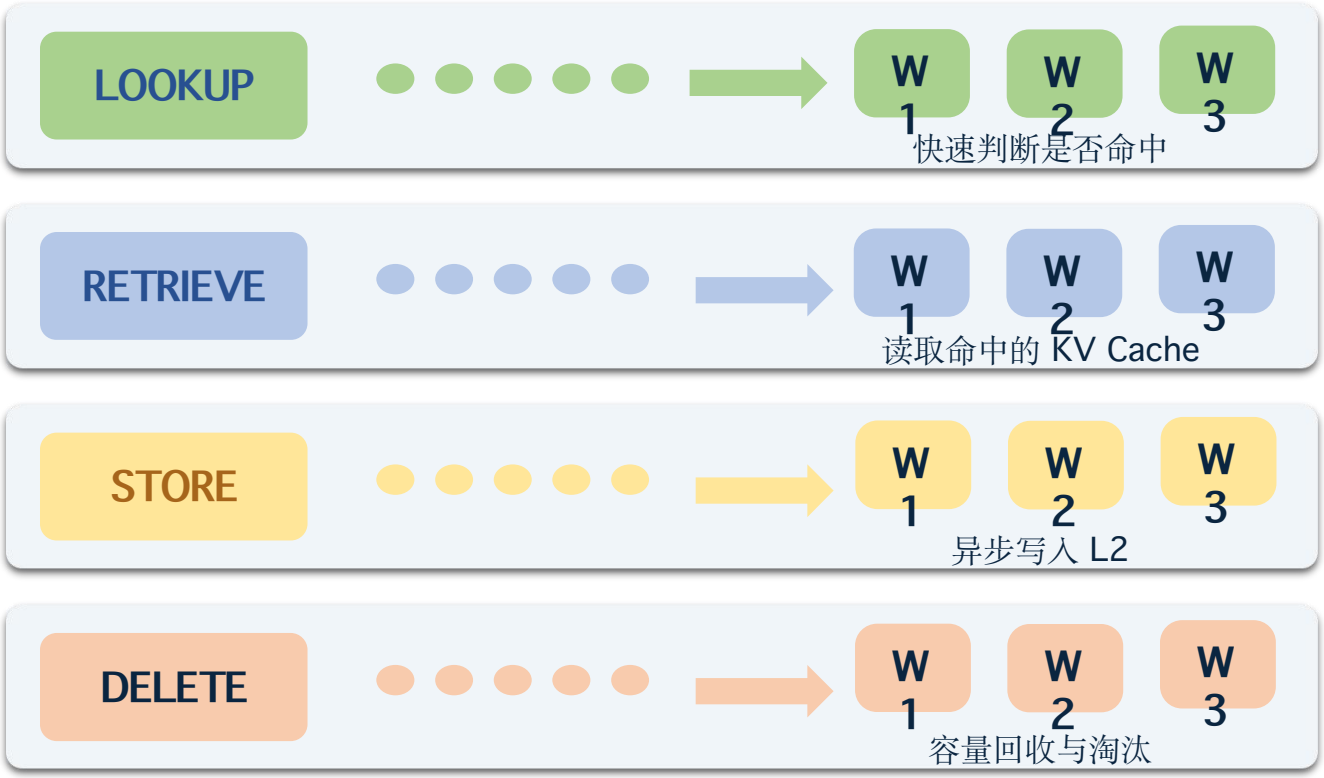
AEROSPIKE

Aerospike

L2 Adapter 可作为独立包维护，统一异步接口隔离后端差异

Native Adapter：按操作隔离并发

03 可插拔扩展与多平台



按操作分配线程池

lookup / retrieve / store / delete
分别配置

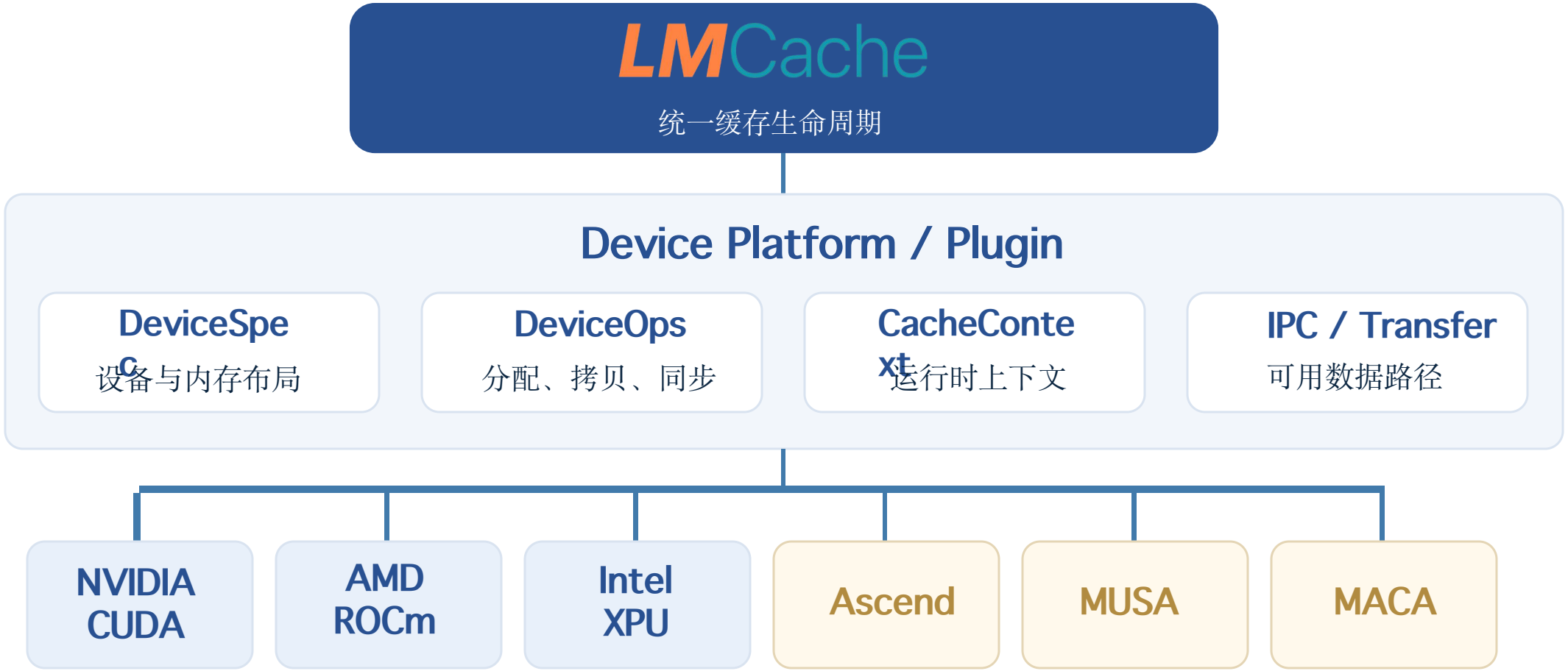
批处理与完成通知

Batch tiling / completion queue
eventfd 唤醒

避免慢写入阻塞命中判断，同时测量队列等待与后端尾延迟

Device Platform：封装硬件差异

03 可插拔扩展与多平台



插件可以独立维护；各平台分别验证内存、IPC、传输与同步能力

CONTENTS

05 生产部署与可观测性



Operator：缓存服务的独立部署

04 生产部署与可观测性

01 声明配置

LMCacheEngine

自定义资源 CR

容量	缓存空间规划
端口	服务连接配置
L2	存储后端配置
监控	监控接入配置

02 持续收敛

LMCache Operator

读取 CR 中的期望状态
协调部署对象
持续检查与更新

03 部署输出

DaemonSet + Service
每节点部署 LMCache Server

ConfigMap + Monitor
配置管理与 Prometheus 监控

Webhook
可选注入 vLLM 连接

分别规划缓存容量与引擎副本，验证节点亲和性、资源配置与连接关系

可观测性指标

生产洞察

健康状态

节点状态与核心指标

分层容量

GPU / DRAM 水位

读写异常

错误码与失败链路

引擎拓扑

在线节点 / 版本 / 负载

性能调试

缓存命中

命中趋势与利用率

生命周期

创建 / 命中 / 淘汰

冷热周转

淘汰至复用的间隔

I/O 瓶颈

IOPS / 吞吐 / 延迟

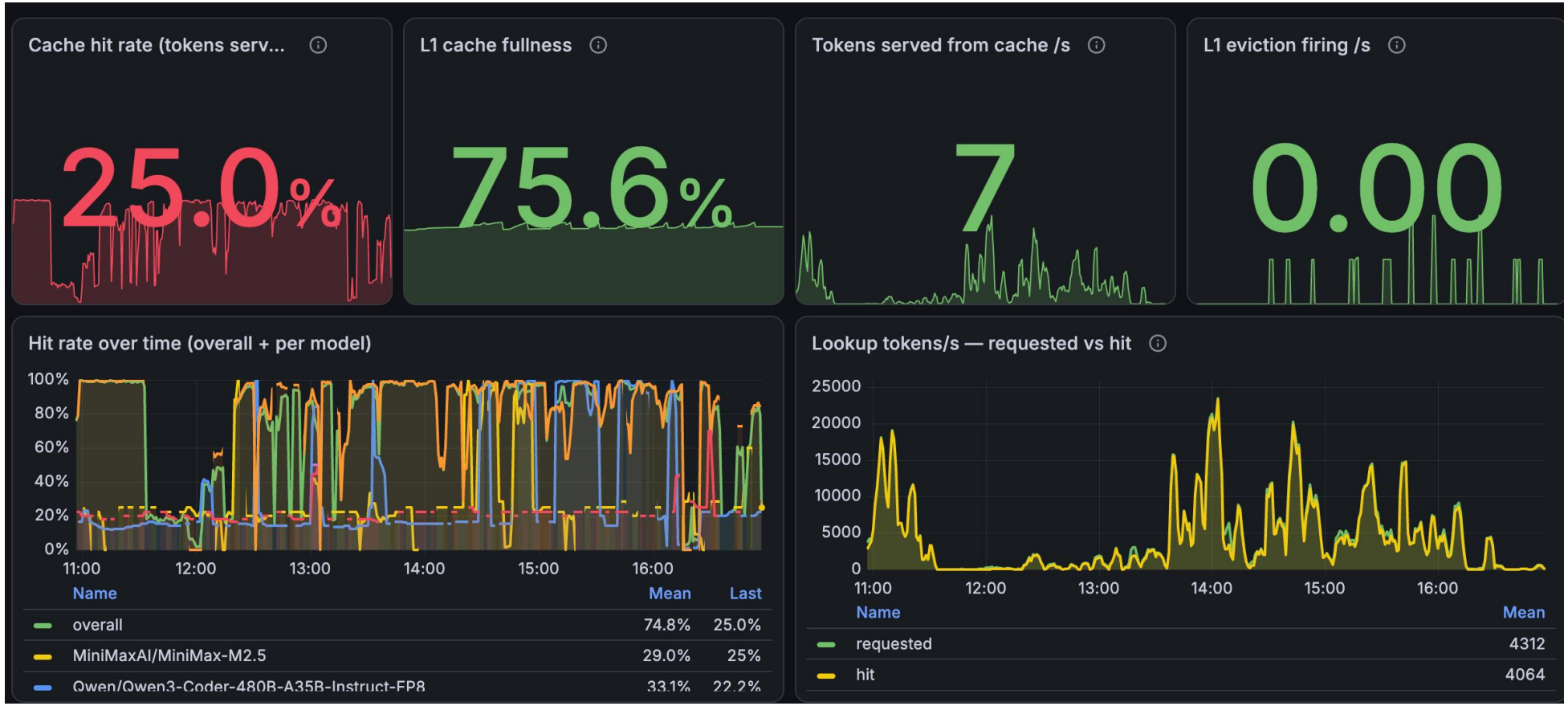
Prometheus + OpenTelemetry

Metrics / Logs / Traces 关联分析

集成业界标准协议，实现Metrics、Logs、Traces的统一采集、存储与关联分析，为高并发生产环境提供高可用、高扩展的监控基础设施支撑。

命中率、缓存水位与请求延迟

04 生产部署与可观测性



截图为原材料监控示例；命中增加仍需结合加载延迟解释请求收益

故障隔离与恢复验证

04 生产部署与可观测性

01 vLLM 主进程崩溃

vLLM 进程	LMCache 进程
✕ 异常退出	√ 独立运行

缓存继续驻留

恢复路径：引擎重启 ➡ 注册后复用 KV

02 LMCache 服务宕机

vLLM 进程	LMCache 进程
√ 降级继续推理	✕ 超时 / 不可用

缓存服务恢复

恢复路径：重新连接 ➡ 恢复共享缓存

进程隔离 + 故障降级 + 恢复验证

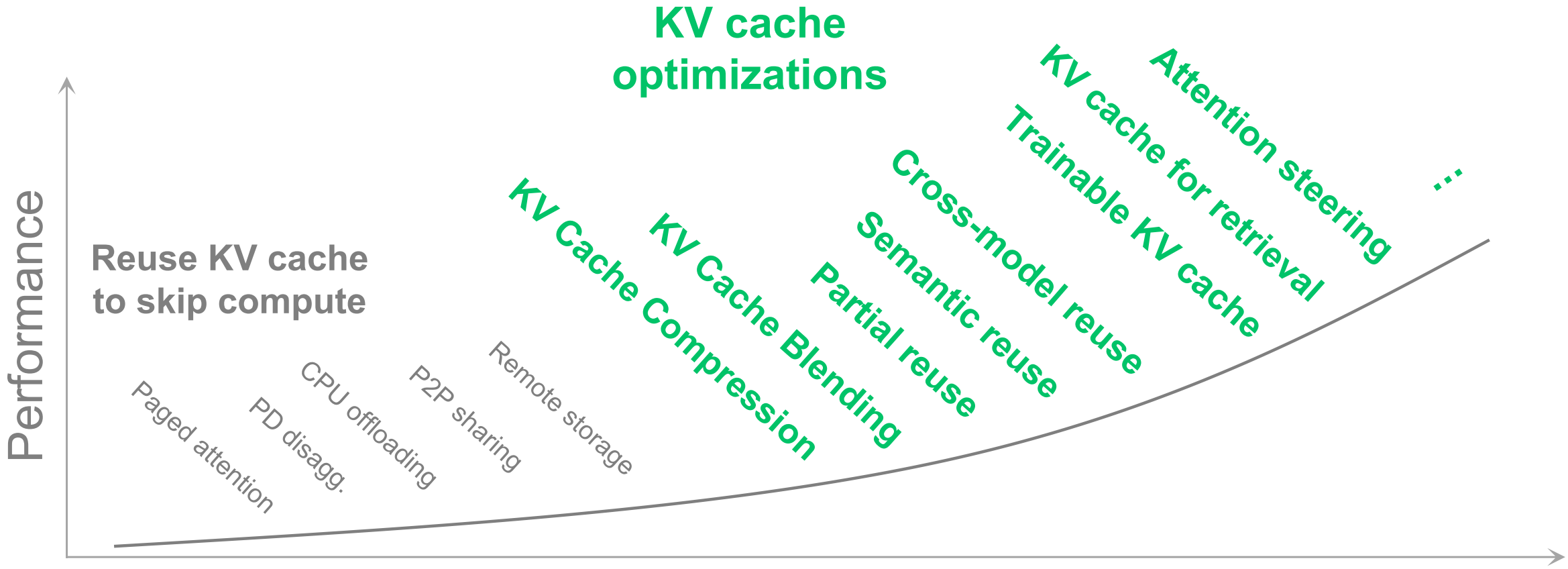
KV 复用需满足兼容条件；降级与重连行为按版本、配置及故障测试确认

CONTENTS

06 社区与未来方向



06 社区与未来方向



LMCache 将 KV Cache 从单机状态升级为可跨实例、跨节点复用的分布式缓存基础设施。

社区现状

06 社区与未来方向

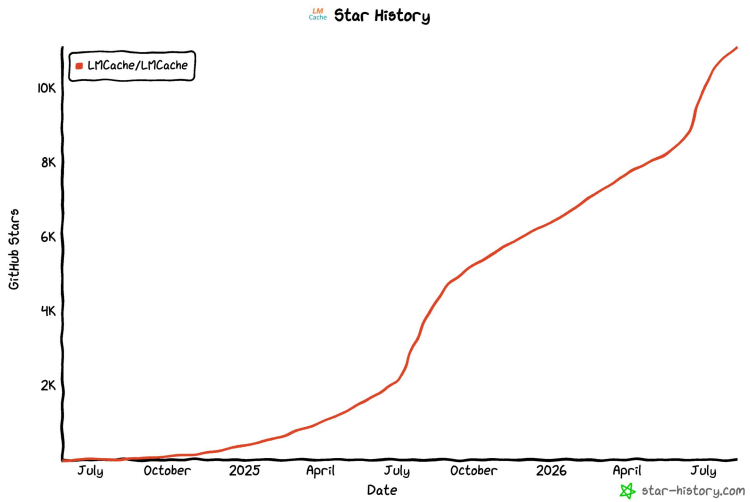
12K+	380+	1.05M+	1200+
GitHub Stars	贡献者	PyPI 下载量	社区成员

一起参与 LMCache共建

github.com/LMCache/LMCache

Star　Fork　贡献

GitHub Star 增长



贡献类型	随 PR 提交的材料
修复与文档	版本与配置 / 最小复现 预期行为 / 验证步骤
插件与集成	接口与兼容边界 真实模型测试 / 使用示例
性能与架构	负载与对照基线 资源开销 / 尾延迟回归

PR 评审

可复现的验证

github.com/LMCache/LMCache / Issues & Contributing

GOSIM

GOSIM Shenzhen 2026



群聊：LMCache 开源爱好者 3 群

Valid until 10/19 and will update upon joining group

Thanks!